

数据库与流程实现

核心业务流程与数据库交互说明

一、文档目的

本文档用于说明项目中主要业务流程的实现思路，重点描述：

- 业务场景与参与角色
- 前后端在流程中的交互方式
- 后端对数据库表执行的增删改查操作
- 各表之间的逻辑关联
- 状态流转与事务边界

二、系统中的核心角色

系统将“身份”和“权限”分开建模。

2.1 身份类型

存放在 `users.user_type` 中：

- `student`
- `teacher`

2.2 权限角色

通过 `roles + user_roles` 管理：

- `BASIC_USER`：普通用户
- `ORGANIZER`：活动负责人，可发起活动、发布招募、审核报名、提交结项
- `REVIEWER`：审核人，可审核立项/结项申请
- `SYS_ADMIN`：系统管理员，可审核权限申请、管理系统

2.3 主要组织关系

通过 `organizations` 和 `user_organizations` 表示：

- 活动发起组织
 - reviewer 所属审核组织
 - 某些通知按组织范围发送
-

三、总览：主要业务流程

当前版本主要包含以下业务流程：

1. 用户注册与登录
2. 普通用户申请权限（ORGANIZER / REVIEWER / SYS_ADMIN）
3. ORGANIZER 发起活动立项申请
4. 系统生成立项审核待办
5. REVIEWER 审核立项申请
6. 立项通过后生成正式活动
7. ORGANIZER 发布活动招募
8. 用户报名活动并提交报名材料
9. ORGANIZER 审核报名
10. ORGANIZER 创建签到场次，用户签到
11. ORGANIZER 提交结项申请
12. REVIEWER 审核结项申请并关闭活动
13. 发布新闻/公告
14. 发送站内通知并记录已读状态

下面按流程分别说明。

四、业务流程详解

4.1 用户注册与登录

4.1.1 业务背景

用户首次使用系统时，需要注册账号。注册时区分为：

- 学生
- 教师

注册后默认只具备普通用户权限，即 `BASIC_USER`。

如果以后要承担负责人、审核人、管理员职责，需要再走权限申请流程。

4.1.2 涉及表

- `users`
 - `student_profiles`
 - `teacher_profiles`
 - `roles`
 - `user_roles`
 - `system_logs`
-

4.1.3 注册流程

前端输入

用户填写：

- 学号/工号 `username`
- 姓名 `real_name`
- 身份类型 `user_type`
- 密码
- 可选邮箱、手机号

后端处理逻辑

1. 检查 `users.username` 是否已存在
2. 对密码做哈希处理
3. 向 `users` 插入一条记录
4. 根据 `user_type` 创建对应 profile
 - `student` → `student_profiles`
 - `teacher` → `teacher_profiles`
5. 查询 `roles` 表中 `BASIC_USER` 的角色 ID

6. 向 `user_roles` 插入一条 `(user_id, BASIC_USER)` 记录

7. 写入 `system_logs`

4.1.4 数据库操作

写入

- `INSERT users`
- `INSERT student_profiles` 或 `INSERT teacher_profiles`
- `INSERT user_roles`
- `INSERT system_logs`

查询

- `SELECT users WHERE username = ?`
 - `SELECT roles WHERE code = 'BASIC_USER'`
-

4.1.5 登录流程

前端输入

- username
- password

后端处理逻辑

1. 查询 `users` 获取账号信息
 2. 校验密码哈希
 3. 校验用户状态 `status = active`
 4. 查询 `user_roles` 和 `roles`
 5. 更新 `users.last_login_at`
 6. 写入 `system_logs`
 7. 返回用户基础信息、身份类型、角色列表
-

4.2 普通用户申请权限

4.2.1 业务背景

普通用户注册后只有 `BASIC_USER`。

如果需要成为：

- 活动负责人 `ORGANIZER`
- 审核人 `REVIEWER`
- 系统管理员 `SYS_ADMIN`

则需要提交权限申请，由现有 `SYS_ADMIN` 审核。

其中：

- 申请 `REVIEWER` 时必须是 `teacher`
 - 申请 `REVIEWER` / `ORGANIZER` 时通常要填写所属组织
-

4.2.2 涉及表

- `role_applications`
 - `pending_tasks`
 - `notifications`
 - `notification_receipts`
 - `user_roles`
 - `user_organizations`
 - `system_logs`
-

4.2.3 提交权限申请流程

前端输入

- 目标角色 `target_role_code`
- 所属组织 `organization_id`（视角色而定）
- 申请理由 `reason`

后端处理逻辑

1. 校验申请人当前身份和角色
2. 若申请 `REVIEWER`，校验 `users.user_type = teacher`
3. 插入 `role_applications`

4. 查询所有 `SYS_ADMIN`
 5. 为对应系统管理员生成待办 `pending_tasks`
 6. 发送通知 `notifications`
 7. 写入 `system_logs`
-

4.2.4 数据库操作

查询

- `SELECT users WHERE id = applicant_id`
- `SELECT user_roles / roles` 判断当前权限
- `SELECT users JOIN user_roles JOIN roles WHERE code = 'SYS_ADMIN'`

写入

- `INSERT role_applications`
 - `INSERT pending_tasks (task_type = role_application_review)`
 - `INSERT notifications`
 - `INSERT system_logs`
-

4.2.5 审核权限申请流程

审核通过时

1. 更新 `role_applications.status = approved`
2. 填写 `reviewed_by / reviewed_at / review_comment`
3. 向 `user_roles` 插入新角色
4. 若目标是 `REVIEWER` 或 `ORGANIZER` 且有组织，插入 `user_organizations`
5. 更新待办 `pending_tasks.status = processed`
6. 向申请人发送审核结果通知
7. 写入系统日志

审核驳回时

1. 更新 `role_applications.status = rejected`
2. 更新待办

3. 发送驳回通知
 4. 写入日志
-

4.3 ORGANIZER 发起活动立项申请

4.3.1 业务背景

拥有 `ORGANIZER` 权限的用户，可以代表某个组织发起活动立项申请。
立项申请是正式活动创建前的前置流程，申请通过后会生成活动。

4.3.2 涉及表

- `activity_applications`
 - `application_attachments`
 - `user_roles`
 - `organizations`
 - `pending_tasks`
 - `notifications`
 - `system_logs`
-

4.3.3 保存草稿流程

前端输入

- 发起组织
- 活动名称
- 简介
- 时间地点
- 申请材料

后端处理逻辑

1. 校验当前用户拥有 `ORGANIZER`
2. 校验所选组织存在且可用
3. 若是首次保存，插入 `activity_applications(status = draft)`

4. 若是编辑草稿，更新 `activity_applications`
 5. 对新上传文件插入 `application_attachments`
-

4.3.4 数据库操作

查询

- `SELECT roles / user_roles` 判断当前用户是否有 `ORGANIZER`
- `SELECT organizations WHERE id = ?`

写入/更新

- `INSERT activity_applications` 或 `UPDATE activity_applications`
 - `INSERT application_attachments`
 - 可选：删除被替换附件、重新上传附件
-

4.3.5 正式提交申请流程

后端处理逻辑

1. 检查申请状态必须是 `draft` 或 `need_more`
 2. 检查必要字段是否齐全
 3. 检查必要附件是否齐全
 4. 更新：
 - `status = submitted`
 - `submitted_at = now`
 5. 启动“动态审核推导”逻辑
 6. 给一级审核人生成待办
 7. 更新申请状态为 `approving`
 8. 给申请人发送“已提交”通知
 9. 写入系统日志
-

4.4 系统生成立项审核待办（动态审核链）

4.4.1 业务背景

当前版本不为每个组织手工配置完整审核链，而是根据：

- 发起组织类型
- 上级组织关系
- reviewer 所属组织

动态推导审核路径。

4.4.2 涉及表

- `activity_applications`
 - `organizations`
 - `user_organizations`
 - `user_roles`
 - `roles`
 - `pending_tasks`
 - `notifications`
-

4.4.3 推导逻辑示例

当前规则示例：

1. **社团 (club) 发起活动**
 - 一级审核组织：社团指导中心
 - 二级审核组织：学工部
 2. **学生组织 (student_organization) 发起活动**
 - 一级审核组织：对应学院团委
 - 二级审核组织：校团委
 3. **学院团委 (administration 中的学院团委) 发起活动**
 - 一级审核组织：校团委
 - 二级审核组织：校党委宣传部
-

4.4.4 数据库交互

查询

1. `SELECT organizations WHERE id =`
2. 根据组织类型和 `parent_org_id` 逐级查找上级组织
3. 查询该审核组织下的 reviewer:
 - `user_organizations`
 - `user_roles`
 - `roles.code = REVIEWER`
 - `users.user_type = teacher`

写入/更新

1. 更新 `activity_applications.current_level = 1`
 2. 更新 `activity_applications.current_reviewer_id = reviewer_id`
 3. 更新 `activity_applications.status = approving`
 4. 插入 `pending_tasks`
 - `task_type = application_review`
 - `related_resource_type = activity_application`
 - `related_resource_id = activity_applications.id`
 5. 插入通知给 reviewer
-

4.5 REVIEWER 审核立项申请

4.5.1 业务背景

reviewer 在待办列表中处理立项申请，审核结果有三种：

- `approved`
 - `rejected`
 - `need_more`
-

4.5.2 涉及表

- `pending_tasks`
- `activity_applications`

- `approval_records`
 - `activities`
 - `notifications`
 - `system_logs`
-

4.5.3 审核通过但不是最后一级

后端处理逻辑

1. 写入 `approval_records`
 2. 更新当前待办为 `processed`
 3. 推导下一级审核组织
 4. 找到下一级 reviewer
 5. 更新 `activity_applications.current_level`
 6. 更新 `current_reviewer_id`
 7. 生成新的 `pending_tasks`
 8. 给申请人发送阶段性通知
 9. 给下一级 reviewer 发送待办提醒
-

4.5.4 审核通过且为最后一级

后端处理逻辑

1. 写入最终 `approval_records`
2. 更新待办为 `processed`
3. 更新 `activity_applications.status = approved`
4. 清空 `current_level/current_reviewer_id`
5. 创建 `activities`
6. 给申请人发送“立项通过”通知
7. 写入系统日志

数据库写入

- `INSERT approval_records`

- `UPDATE pending_tasks`
 - `UPDATE activity_applications`
 - `INSERT activities`
 - `INSERT notifications`
-

4.5.5 审核驳回

后端处理逻辑

1. 写入 `approval_records(result = rejected)`
 2. 更新待办为 `processed`
 3. 更新 `activity_applications.status = rejected`
 4. 发送驳回通知
-

4.5.6 审核要求补材料

后端处理逻辑

1. 写入 `approval_records(result = need_more)`
2. 更新待办
3. 更新 `activity_applications.status = need_more`
4. 通知申请人补材料

申请人补充后，可再次编辑并重新提交同一条申请。

4.6 立项通过后生成正式活动

4.6.1 业务背景

立项申请通过后，系统生成正式活动。

后续所有活动执行流程都围绕 `activities` 展开。

4.6.2 涉及表

- `activities`

- `activity_applications`
-

4.6.3 数据库交互

创建活动表时，通常把申请表中的核心信息复制到活动表：

- `application_id`
- `title`
- `organizer_id`
- `organization_id`
- `start_time`
- `end_time`
- `status = planned`
- `created_at`

说明

`activities` 相当于“审核通过后的正式业务实体”，
而 `activity_applications` 仍保留为历史申请记录。

4.7 ORGANIZER 发布活动招募

4.7.1 业务背景

活动立项后，负责人可发布招募，控制报名时间、人数、可报名对象。

4.7.2 涉及表

- `activities`
 - `recruitments`
 - `recruitment_allowed_majors`
 - `announcements`
 - `system_logs`
-

4.7.3 创建招募草稿

后端操作

1. 校验当前用户是该活动的 `organizer_id`
 2. 校验活动状态允许发布招募（通常 `planned / recruiting`）
 3. 插入 `recruitments(status = draft)`
 4. 若设置专业限制，插入 `recruitment_allowed_majors`
-

4.7.4 发布招募

后端操作

1. 更新 `recruitments.status = published`
 2. 若活动原状态为 `planned`，则将 `activities.status = recruiting`
 3. 可选：创建一条招募宣传公告 `announcements`
 4. 写入系统日志
-

4.8 用户报名活动

4.8.1 业务背景

普通用户报名时，系统要根据招募限制检查其是否符合条件。

4.8.2 涉及表

- `users`
- `student_profiles`
- `recruitments`
- `recruitment_allowed_majors`
- `recruitment_signups`
- `signup_attachments`
- `pending_tasks`

- notifications
-

4.8.3 报名前校验

查询

1. `SELECT recruitments`
 2. 检查招募状态是否为 `published`
 3. 检查报名时间是否在 `signup_start_time ~ signup_end_time`
 4. 检查 `target_user_type`
 5. 若用户是学生：
 - 读取 `student_profiles`
 - 检查 `min_grade/max_grade`
 - 检查 `recruitment_allowed_majors`
-

4.8.4 创建报名记录

写入

1. `INSERT recruitment_signups(status = pending, applied_at = now)`
 2. 若需要材料，插入 `signup_attachments`
 3. 为活动负责人生成待办：
 - `task_type = signup_review`
 - `related_resource_type = recruitment_signup`
 4. 给报名人发送“报名提交成功”通知
 5. 给 organizer 发送“有新报名待审核”通知
-

4.9 ORGANIZER 审核报名

4.9.1 业务背景

报名审核不是行政审批，而是活动内部筛选，因此由该活动负责人处理。

4.9.2 涉及表

- `recruitment_signups`
 - `activities`
 - `pending_tasks`
 - `notifications`
 - `system_logs`
-

4.9.3 审核通过

后端操作

1. 更新 `recruitment_signups.status = approved`
 2. 更新 `reviewed_at / reviewed_by`
 3. 更新对应待办为 `processed`
 4. 给报名人发送“报名通过”通知
 5. 写入日志
-

4.9.4 审核拒绝

后端操作

1. 更新 `recruitment_signups.status = rejected`
 2. 更新审核人信息
 3. 更新待办
 4. 给报名人发送“报名未通过”通知
-

4.10 ORGANIZER 创建签到场次，用户签到

4.10.1 业务背景

活动开始前后，负责人可以创建签到场次。

签到方式支持：

- 二维码

- 签到码
 - 手动签到
-

4.10.2 涉及表

- `checkin_sessions`
 - `checkin_records`
 - `activities`
 - `system_logs`
-

4.10.3 创建签到场次

后端操作

1. 校验当前用户是活动负责人或具备 `organizer` 权限
 2. 插入 `checkin_sessions(status = pending)`
 3. 可生成 `checkin_code` 或 `qrcode_token`
 4. 若活动状态为 `planned / recruiting`，并执行“开启活动/开启签到”，则将活动改为 `ongoing`
-

4.10.4 用户签到

后端操作

1. 校验签到场次状态为 `open`
 2. 校验当前时间在签到窗口内
 3. 校验用户是否有资格参加该活动
 4. 插入 `checkin_records`
 - `status = checked_in` 或 `late`
 5. 避免重复签到（依赖 `UNIQUE(session_id, user_id)`）
-

4.11 ORGANIZER 提交结项申请

4.11.1 业务背景

活动结束后，负责人提交结项申请，上传总结和材料，进入结项审核流程。

4.11.2 涉及表

- `closure_applications`
 - `closure_attachments`
 - `activities`
 - `pending_tasks`
 - `notifications`
 - `system_logs`
-

4.11.3 提交流程

后端操作

1. 校验当前用户是该活动负责人
 2. 若首次创建，则插入 `closure_applications(status = draft)`
 3. 上传材料到 `closure_attachments`
 4. 正式提交时：
 - 更新 `status = submitted`
 - 填写 `submitted_at`
 5. 按与立项类似的规则推导结项审核组织
 6. 给一级 reviewer 生成 `closure_review` 待办
 7. 更新结项申请状态为 `approving`
 8. 给申请人和 reviewer 发送通知
-

4.12 REVIEWER 审核结项申请并关闭活动

4.12.1 业务背景

结项审核与立项审核类似，也是多级 reviewer 审核。

审核通过后，活动正式关闭。

4.12.2 涉及表

- `closure_applications`
 - `closure_review_records`
 - `pending_tasks`
 - `activities`
 - `notifications`
 - `system_logs`
-

4.12.3 审核处理

若不是最后一级

1. 写入 `closure_review_records`
2. 更新当前待办
3. 推导下一级 reviewer
4. 生成下一条待办
5. 更新结项申请当前状态

若是最后一级且通过

1. 写入最终审核记录
2. 更新 `closure_applications.status = approved`
3. 更新待办
4. 更新 `activities.status = closed`
5. 给 organizer 发送“结项通过”通知

若驳回/补充

1. 写入审核记录
 2. 更新 `closure_applications.status`
 3. 更新待办
 4. 通知申请人
-

4.13 发布新闻/公告

4.13.1 业务背景

系统中公告分为：

- 新闻
- 公告
- 招募宣传
- 系统公告

除系统公告外，通常都应与某个活动关联。

4.13.2 涉及表

- `announcements`
 - `activities`
 - `system_logs`
-

4.13.3 发布流程

后端操作

1. 校验发布人权限
 2. 若是活动相关公告，校验 `activity_id` 是否存在
 3. 校验活动状态是否允许当前类型公告
 - 招募发布： `planned / recruiting`
 - 过程宣传： `ongoing`
 - 总结宣传： `finished`
 4. 插入或更新 `announcements`
 5. 若正式发布，更新 `status = published`、写入 `published_at`
 6. 写入系统日志
-

4.14 发送通知与已读记录

4.14.1 业务背景

通知用于向特定用户发送流程结果和提醒。

4.14.2 涉及表

- `notifications`
 - `notification_receipts`
-

4.14.3 创建通知

系统在以下场景创建通知：

- 立项申请提交成功
- 立项审核结果
- 报名提交成功
- 报名审核结果
- 结项审核结果
- 权限申请审核结果
- 新待办提醒
- 活动开始/截止提醒

写入

- `INSERT notifications`

其中：

- `target_type` 决定发给谁
 - `target_value` 存对应值
 - `source_type/source_id` 指向来源业务数据
-

4.14.4 生成接收记录

如果系统采用“预生成已读记录”的实现方式，则在发送通知时：

1. 根据 `target_type + target_value` 解析目标用户集合
 2. 向 `notification_receipts` 批量插入每个用户的接收记录
 3. 默认 `is_read = false`
-

4.14.5 用户查看通知

后端操作

1. 查询当前用户的 `notification_receipts`
2. 关联 `notifications`
3. 返回消息列表

标记已读

1. 更新 `notification_receipts.is_read = true`
 2. 更新 `read_at = now`
-

五、统一待办表的作用

`pending_tasks` 是系统中的**统一任务入口表**。

它不保存复杂业务字段，而是通过：

- `task_type`
- `related_resource_type`
- `related_resource_id`

指向真正的业务主表。

5.1 当前支持的待办类型

- `application_review` ↔ `activity_application`
- `closure_review` ↔ `closure_application`
- `signup_review` ↔ `recruitment_signup`
- `role_application_review` ↔ `role_application`

5.2 设计意义

这样可以让前端统一展示“我的待办”，而不需要为每种业务做一套完全不同的待办表。

六、表之间最重要的逻辑链

6.1 立项链

activity_applications
→ application_attachments
→ approval_records
→ activities

6.2 招募链

activities
→ recruitments
→ recruitment_allowed_majors
→ recruitment_signups
→ signup_attachments

6.3 签到链

activities
→ checkin_sessions
→ checkin_records

6.4 结项链

activities
→ closure_applications
→ closure_attachments
→ closure_review_records

6.5 权限申请链

role_applications
→ pending_tasks
→ user_roles
→ user_organizations

6.6 消息链

notifications
→ notification_receipts

七、实现建议：哪些操作应放进事务

以下操作建议放进数据库事务中：

7.1 提交立项申请

- 更新申请状态
- 保存附件
- 推导审核人
- 创建待办
- 创建通知

7.2 审核立项申请

- 写审核记录
- 更新申请状态
- 更新/关闭当前待办
- 生成下一待办或创建活动
- 创建通知

7.3 用户报名

- 创建报名记录
- 保存报名材料
- 创建 organizer 待办
- 创建通知

7.4 审核权限申请

- 更新申请状态
- 写入用户角色
- 写入用户组织归属
- 更新待办
- 创建通知

7.5 结项审核通过

- 写审核记录
- 更新结项申请

- 更新活动状态为 `closed`
 - 更新待办
 - 创建通知
-

八、阅读建议

不同分工可以重点阅读的部分如下：

- **前端开发**：重点看各流程的状态变化、列表展示与表单提交时机
- **后端开发**：重点看各业务流程的数据库读写顺序和事务边界
- **测试与部署**：重点看状态流转、权限约束、异常路径
- **需求设计与代码审查**：重点看流程是否闭环、角色职责是否一致
- **文档撰写**：重点看“业务背景—数据库交互—结果状态”的叙述方式
- **项目管理**：重点看哪些流程优先打通，哪些功能可后续扩展